Network Reconnaissance and Risk-Based Vulnerability Assessment

A chronological investigation of an isolated Ubuntu/MySQL laboratory using Nmap

Prepared by: Ajibade Michael

Assessment window: 7–16 October 2026 | Report revised: 17 October 2026

Classification: Educational proof-of-concept, authorised and deliberately isolated VMware environment

Status: Public-release edition | Authorised isolated lab | Synthetic MedTechCare data

Executive summary

This report traces an authorised security assessment from the first Kali network checks through host discovery, full TCP enumeration, service and operating-system identification, targeted Nmap scripts, local Ubuntu/MySQL validation and risk prioritisation. It follows the testing sequence so the reader can see how evidence from one stage informed the next.

The Ubuntu target at 192.168.249.131 exposed TCP/22 (SSH) and TCP/3306 (MySQL) to Kali (192.168.249.132) inside the isolated VMware subnet. The installed OpenSSH package (1:8.9p1-3ubuntu0.16) is newer than Ubuntu's fixed package for CVE-2024-6387 (1:8.9p1-3ubuntu0.10). MySQL was bound to all IPv4 interfaces, UFW was inactive, and a purposefully configured dbadmin@% account had extensive privileges on two lab databases. These observations support a configuration-exposure scenario, not a demonstrated unauthorised database compromise.

Crucial context: The MedTechCare database and dbadmin account were established intentionally for an earlier MSc dissertation proof of concept. The database contains synthetic training data only; no real patient information was used. The risk discussion considers what similar access controls would mean in a real organisation, while recognising that this isolated lab was designed for testing.

Assessment snapshot

Element	Verified detail
Attacker/test VM	Kali Linux 2025.1a; Nmap 7.95; 192.168.249.132
Target VM	Ubuntu 22.04.5 LTS; 192.168.249.131
Lab scope	192.168.249.0/24; educational, authorised; internet intentionally unavailable
TCP exposure	22/tcp OpenSSH, 3306/tcp MySQL; 65,533 other TCP ports closed
Proof of exploit	None attempted or claimed; only authorised checks
Primary evidence	Seven original Nmap .txt output files, local command screenshots

1. Scope, authorisation and safety boundary

Both virtual machines were under the assessor's control. Tests targeted the private VMware lab and did not attempt access to external systems. Network isolation was intentionally retained throughout the exercise. The MedTechCare dataset contained synthetic training records only. The work included observation, port

scanning, protocol enumeration and read-only local configuration queries; it did not include password guessing, remote exploitation, access to real patient records or changes to system settings.

The Ubuntu host's lack of access to public update servers is therefore a testing constraint, not a defect to "fix" during the assessment. External vendor documentation can be consulted from the host computer without connecting the laboratory VMs to the internet.

2. Chronological testing: from setup to risk analysis

The sequence below follows the actual assessment progression. Screenshots are placed immediately beside the relevant explanation, while unmodified Nmap terminal output is retained in separate files for auditability.

Step 01 — Confirm Kali address and Nmap version

Purpose. Identify the assessment machine, its lab interface and the version of the scanning tool.

```
ip addr
nmap --version
```

Observed result. Kali showed 192.168.249.132/24 on its lab-facing interface and Nmap version 7.95.

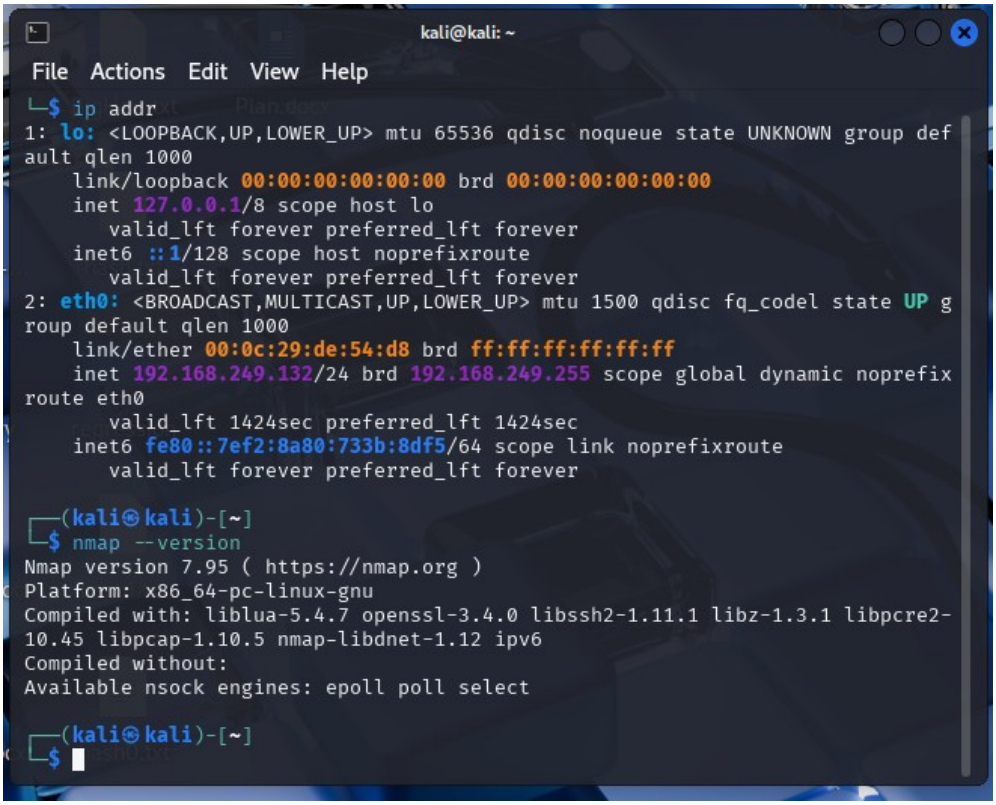


Figure 1. Kali interface identification and Nmap version verification.

Interpretation. The /24 prefix places Kali on the 192.168.249.0/24 lab network; version information makes the work reproducible.

Why next? Identify the Ubuntu target and confirm that local network communication is possible.

Step 02 — Discover systems on the isolated subnet

Purpose. Identify responding hosts without running a full port scan against them.

```
sudo nmap -sn 192.168.249.0/24 -oN ~/nmap-network-scanning-lab/scans/host-discovery.txt
```

Observed result. The saved result lists four responding addresses: 192.168.249.1, 192.168.249.131, 192.168.249.132 and 192.168.249.254.

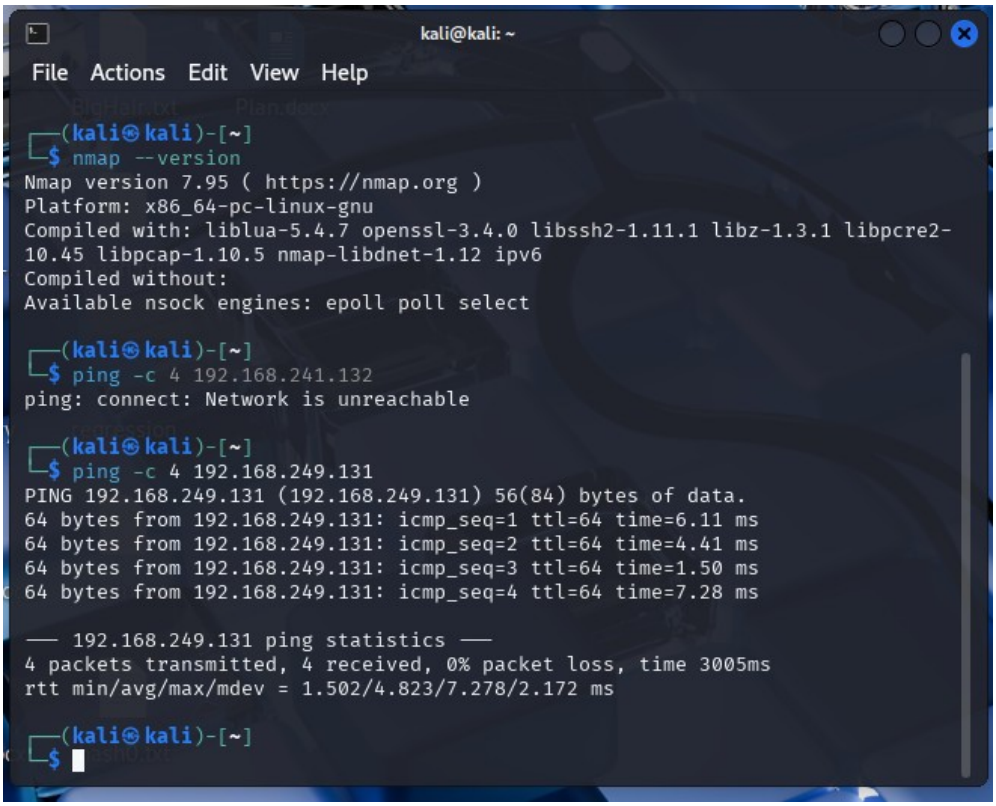


Figure 2. Host discovery on 192.168.249.0/24.

Interpretation. The Ubuntu target was confirmed at 192.168.249.131. The additional VMware-associated addresses were observed, not selected as scan targets for further testing.

Why next? Focus port enumeration on the authorised Ubuntu target.

Saved primary evidence. scans/host-discovery.txt

Step 03 — Identify exposed ports with Nmap's default TCP scan

Purpose. Quickly identify common TCP services listening on the Ubuntu machine.

```
sudo nmap 192.168.249.131
```

Observed result. The default scan reported SSH on 22/tcp and MySQL on 3306/tcp; 998 of the 1,000 default TCP ports were closed.

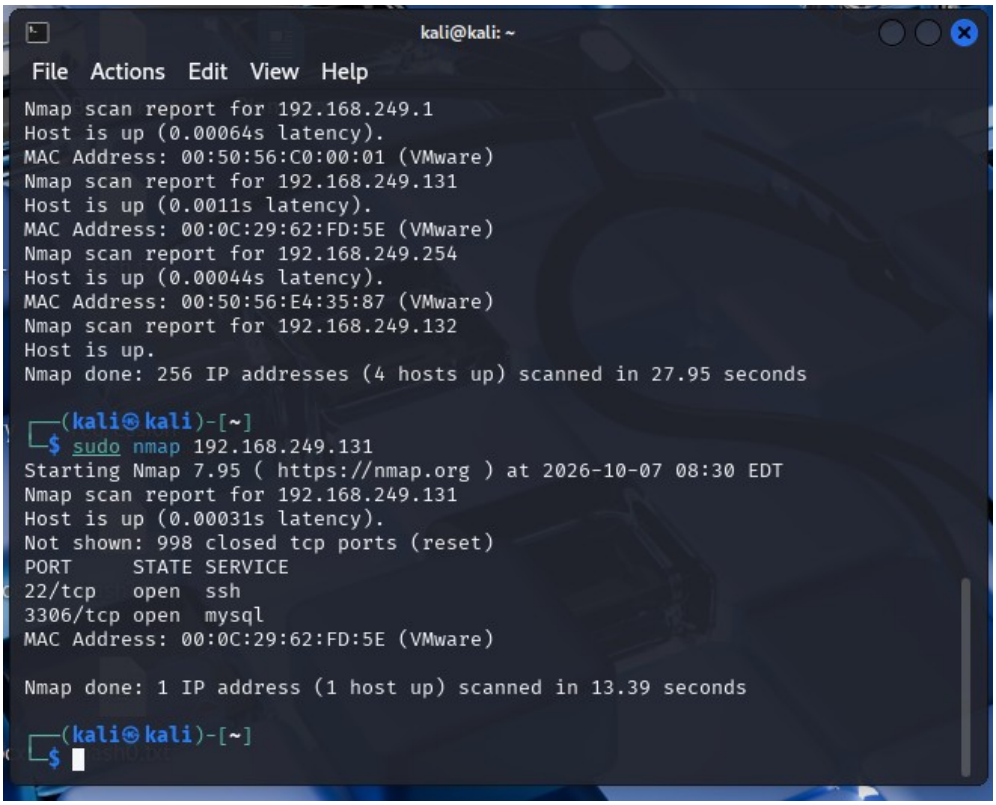


Figure 3. Initial default TCP scanning results.

Interpretation. Open ports show reachable TCP listeners. They do not, by themselves, establish security vulnerabilities.

Why next? Use a full-range scan to check whether other TCP listeners exist outside Nmap's default port selection.

Step 04 — Scan all 65,535 TCP ports

Purpose. Confirm whether the default scan missed services on unusual ports.

```
sudo nmap -p- 192.168.249.131 -oN ~/nmap-network-scanning-lab/scans/full-tcp-scan.txt
```

Observed result. The complete TCP scan showed only ports 22 and 3306 open; 65,533 TCP ports were closed.

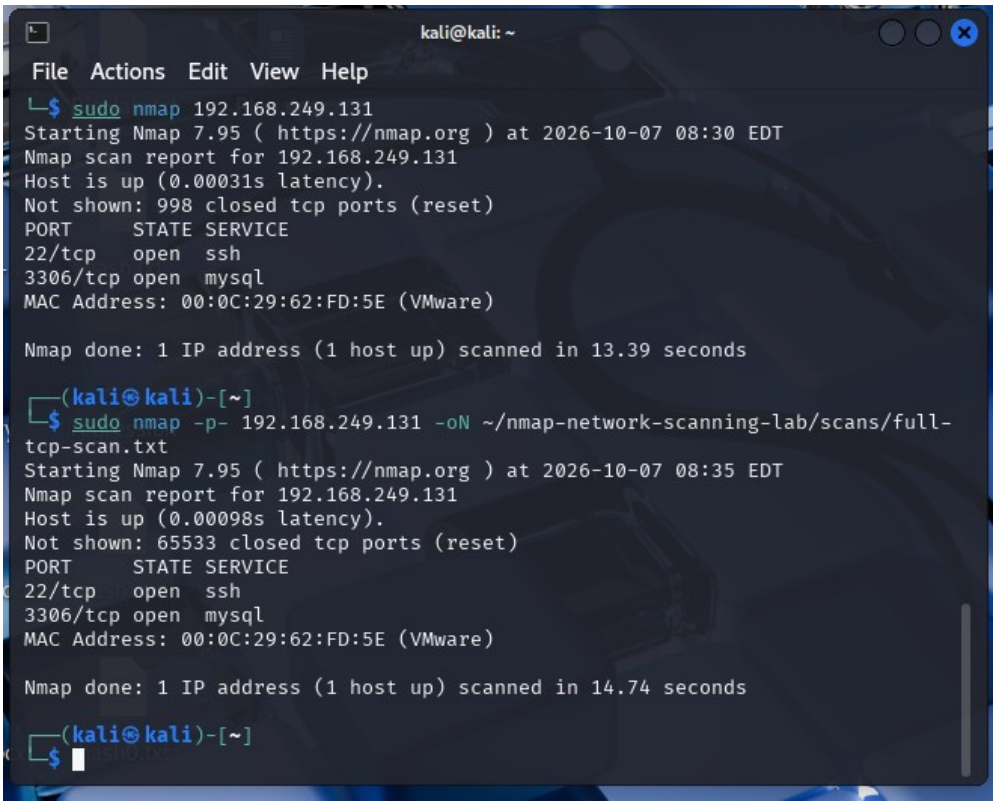


Figure 4. Saved full TCP port scan confirming SSH and MySQL only.

Interpretation. This provides stronger evidence of the TCP exposure surface than the default 1,000-port scan. It does not assess UDP ports.

Why next? Determine which software versions run on the two open services.

Saved primary evidence. scans/full-tcp-scan.txt

Step 05 — Identify SSH and MySQL versions

Purpose. Obtain a version-level fingerprint of each exposed TCP service.

```
sudo nmap -sV -p 22,3306 192.168.249.131 -oN ~/nmap-network-scanning-lab/scans/service-version-detection.txt
```

Observed result. Nmap identified OpenSSH 8.9p1 Ubuntu 3ubuntu0.16 on port 22 and MySQL 8.0.46-0ubuntu0.22.04.3 on port 3306.



Figure 5. SSH and MySQL service fingerprints from the Kali terminal.

Interpretation. Service version strings provide a useful lead for targeted checks; Linux distributors can backport security patches without changing the upstream software version.

Why next? Collect OS context and inspect service-specific configuration evidence before deciding on risk.

Saved primary evidence. scans/service-version-detection.txt

Step 06 — Review Nmap operating-system fingerprints

Purpose. Estimate the target operating-system family and compare Nmap guesses with known lab details.

```
sudo nmap -O 192.168.249.131 -oN ~/nmap-network-scanning-lab/scans/os-detection.txt
```

Observed result. Nmap returned Linux kernel-family guesses alongside router/OpenWrt/MikroTik possibilities.



Figure 6. Nmap OS fingerprinting, including its ambiguous guesses.

Interpretation. These are uncertain fingerprint matches, not proof that Ubuntu is running a router OS. The target was independently confirmed as Ubuntu 22.04.5 LTS using lsb_release -a.

Why next? Continue with protocol-specific enumeration for SSH and MySQL.

Saved primary evidence. scans/os-detection.txt

Step 07 — Enumerate SSH cryptographic algorithms

Purpose. Identify which SSH algorithm families the server advertises.

```
sudo nmap -p 22 --script ssh2-enum-algos 192.168.249.131 -oN ~/nmap-network-scanning-lab/scans/ssh-enumeration.txt
```

Observed result. The result listed Curve25519/ECDH and other key exchanges, modern ciphers such as ChaCha20-Poly1305 and AES-GCM, plus some SHA-1 MAC compatibility options.



Figure 7. SSH supported-algorithm enumeration in Kali.

Interpretation. The SHA-1 MAC entries are potential hardening considerations depending on policy and negotiation; their presence is not itself an exploit demonstration.

Why next? Examine the MySQL handshake and its supported authentication protocol.

Saved primary evidence. scans/ssh-enumeration.txt

Step 08 — Enumerate MySQL protocol information

Purpose. Collect basic metadata made available during the initial MySQL handshake.

```
sudo nmap -p 3306 --script mysql-info 192.168.249.131 -oN ~/nmap-network-scanning-lab/scans/mysql-info.txt
```

Observed result. Nmap confirmed MySQL 8.0.46-0ubuntu0.22.04.3, protocol 10 and the advertised caching_sha2_password authentication plugin.



Figure 8. MySQL handshake metadata; transient session salt redacted in public-release evidence.

Interpretation. Unauthenticated handshake information supports inventory and version verification; disclosing the version does not necessarily constitute a significant vulnerability.

Why next? Check for a narrow authentication misconfiguration signal without attempting brute force.

Saved primary evidence. scans/mysql-info.txt (public copy omits ephemeral handshake salt; the original is preserved privately).

Step 09 — Run the limited MySQL empty-password check

Purpose. Look for a positive result from Nmap's mysql-empty-password script.

```
sudo nmap -p 3306 --script mysql-empty-password 192.168.249.131 -oN ~/nmap-network-scanning-lab/scans/mysql-empty-password.txt
```

Observed result. The scan confirmed 3306/tcp open but did not report a positive empty-password finding.



Figure 9. MySQL empty-password NSE test; no positive result reported.

Interpretation. A missing positive script result is not evidence that every account has a strong password or that authentication has been comprehensively assessed.

Why next? Validate package revisions and inspect the actual database host configuration.

Saved primary evidence. scans/mysql-empty-password.txt

Step 10 — Validate OpenSSH directly on the Ubuntu target

Purpose. Check the installed Debian/Ubuntu package revision and running service rather than relying solely on the Nmap banner.

```
dpkg-query -W openssh-server
lsb_release -a
sudo ss -lntp | grep ':22'
```

Observed result. The Ubuntu machine reported openssh-server 1:8.9p1-3ubuntu0.16 and Ubuntu 22.04.5 LTS; sshd listened on IPv4 and IPv6 port 22.

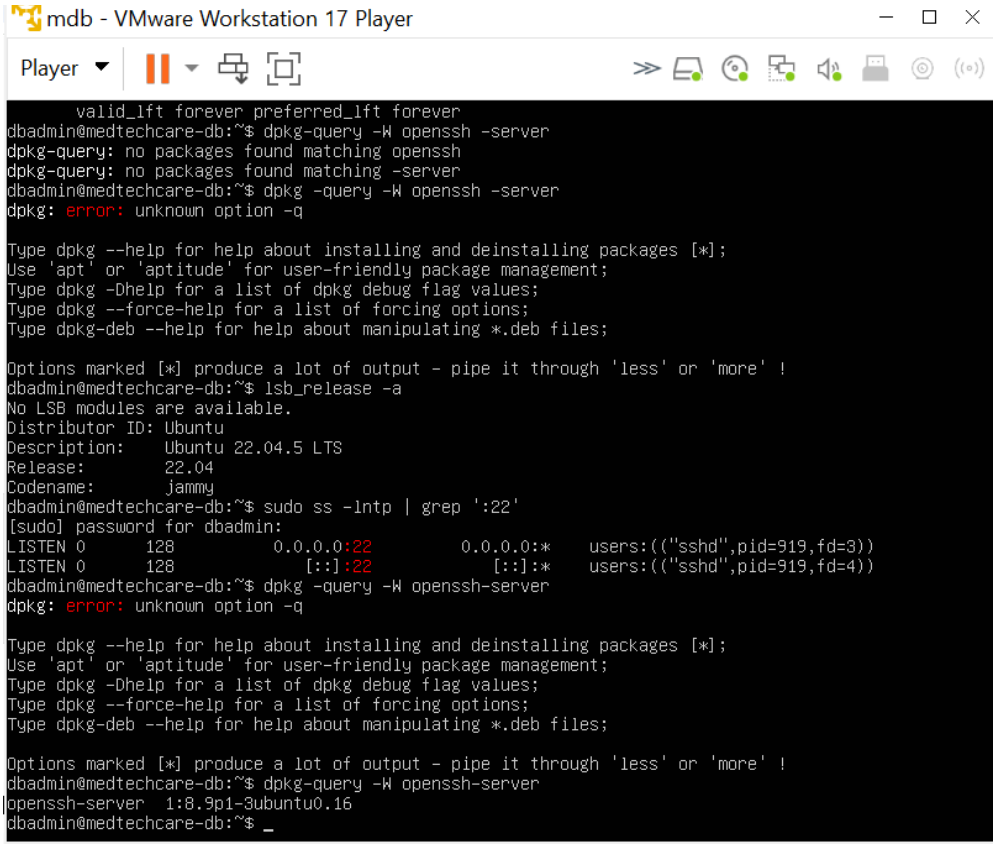


Figure 10. Target-side installed OpenSSH package verification.

Interpretation. Installed package metadata gives a stronger basis for Ubuntu-specific CVE assessment than upstream version alone.

Why next? Compare the installed package with Ubuntu's published remediation release for CVE-2024-6387.

Step 11 — Correlate CVE-2024-6387 with Ubuntu's security notice

Purpose. Verify whether the Nmap-discovered OpenSSH family was actually susceptible to the regreSSHion security issue on this Ubuntu release.

Observed result. Ubuntu's USN-6859-1 states that Ubuntu 22.04 LTS fixes CVE-2024-6387 in openssh-server 1:8.9p1-3ubuntu0.10. The locally installed 1:8.9p1-3ubuntu0.16 is a later Ubuntu revision.

Interpretation. Based on the verified vendor fix and installed package revision, this lab package includes the fix for CVE-2024-6387. No exploit was attempted. This finding does not imply that every other OpenSSH vulnerability is fixed or that the system is fully up to date.

Reference: https://ubuntu.com/security/CVE-2024-6387 and https://ubuntu.com/security/notices/USN-6859-1 (consulted 17 October 2026).

Why next? SSH did not produce a confirmed unpatched finding for that CVE; move to MySQL access controls, where the scan identified a reachable listener.

Step 12 — Check the actual MySQL bind addresses

Purpose. Determine whether the MySQL service is limited to localhost or exposed to the VM network.

```
sudo ss -lntp | grep ':3306'
```

Observed result. The process mysqld listened at 0.0.0.0:3306. The separate MySQL X Protocol listener was bound to 127.0.0.1:33060.

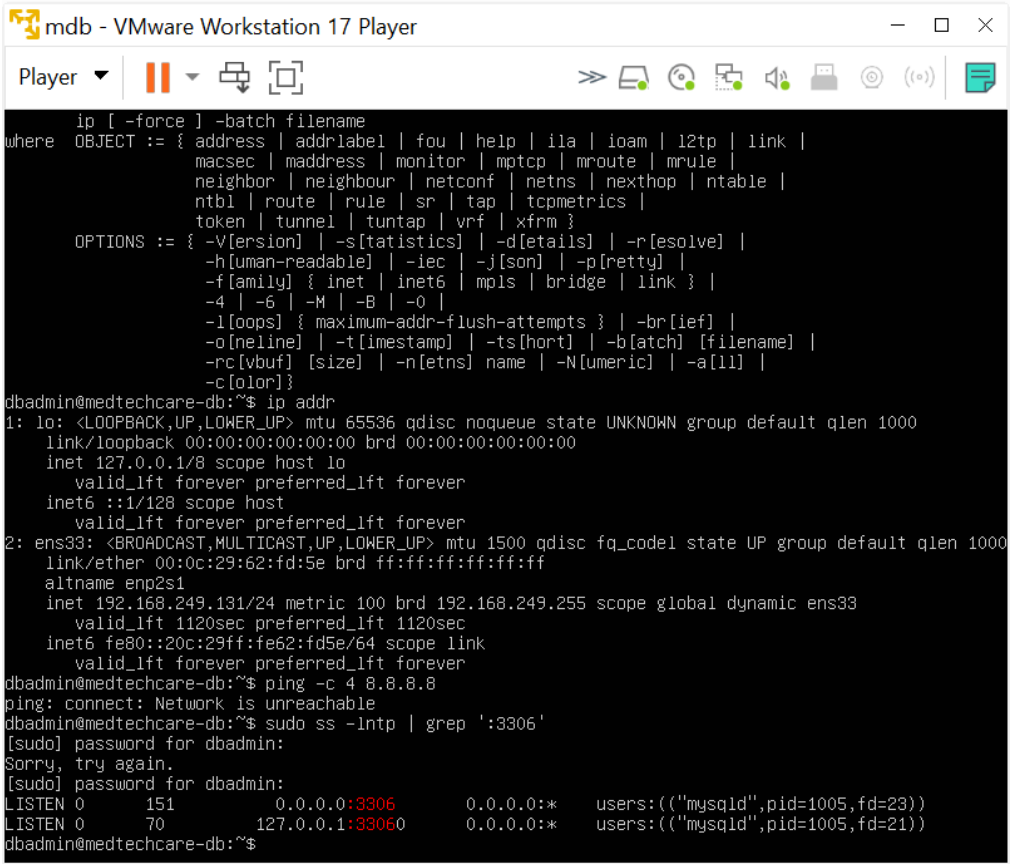


Figure 11. Ubuntu MySQL bind addresses showing classic protocol exposure.

Interpretation. The classic MySQL service accepts connections addressed to any available IPv4 interface; the X Protocol listener shown here is local-only.

Why next? Check whether Ubuntu's firewall filters connections to that network-exposed service.

Step 13 — Check host firewall state

Purpose. Find out whether UFW adds host-level network access controls to the exposed service.

```
sudo ufw status verbose
```

Observed result. UFW returned Status: inactive.

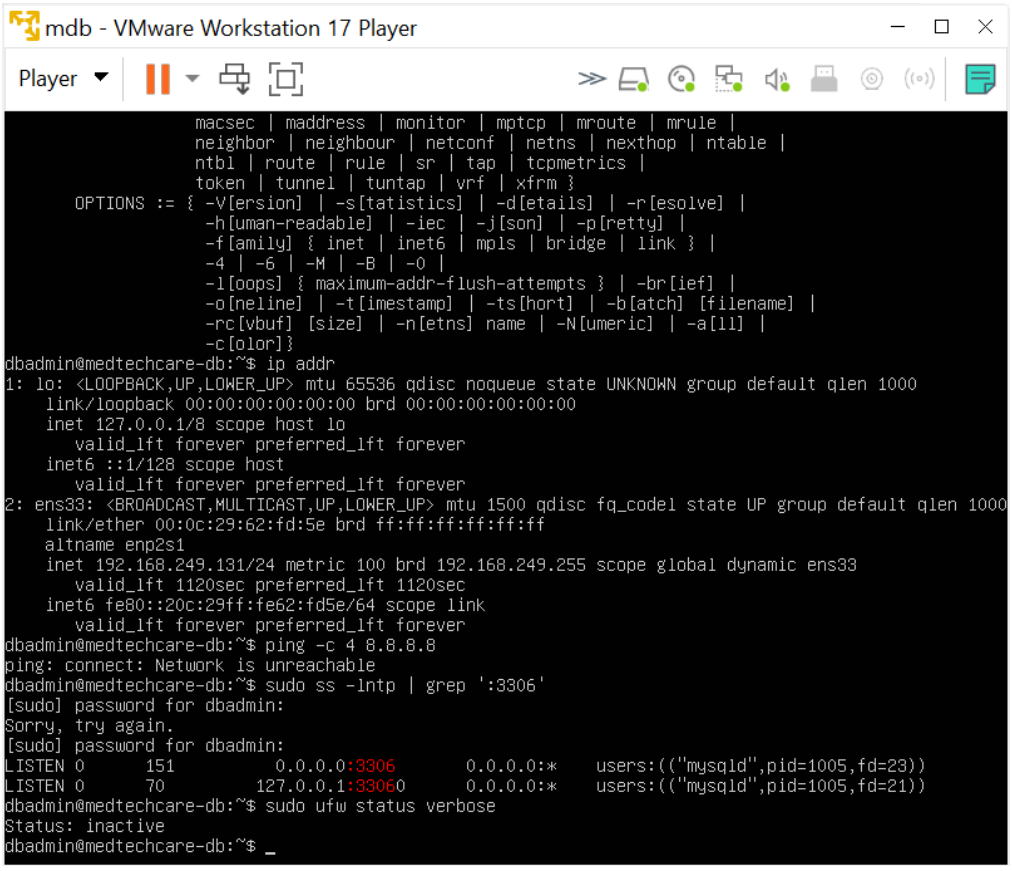


Figure 12. Target-side UFW status.

Interpretation. UFW itself is not enforcing host filtering. This does not rule out other firewall layers or the protective boundary provided by the isolated VMware network.

Why next? Check whether any MySQL accounts accept remote connection attempts.

Step 14 — Review MySQL account host patterns

Purpose. Inspect the permitted client-host patterns and authentication plugins on local database accounts.

```
sudo mysql -e "SELECT user, host, plugin FROM mysql.user;"
```

Observed result. The dbadmin account used host pattern % and authentication plugin mysql_native_password; root was limited to localhost and used auth_socket.

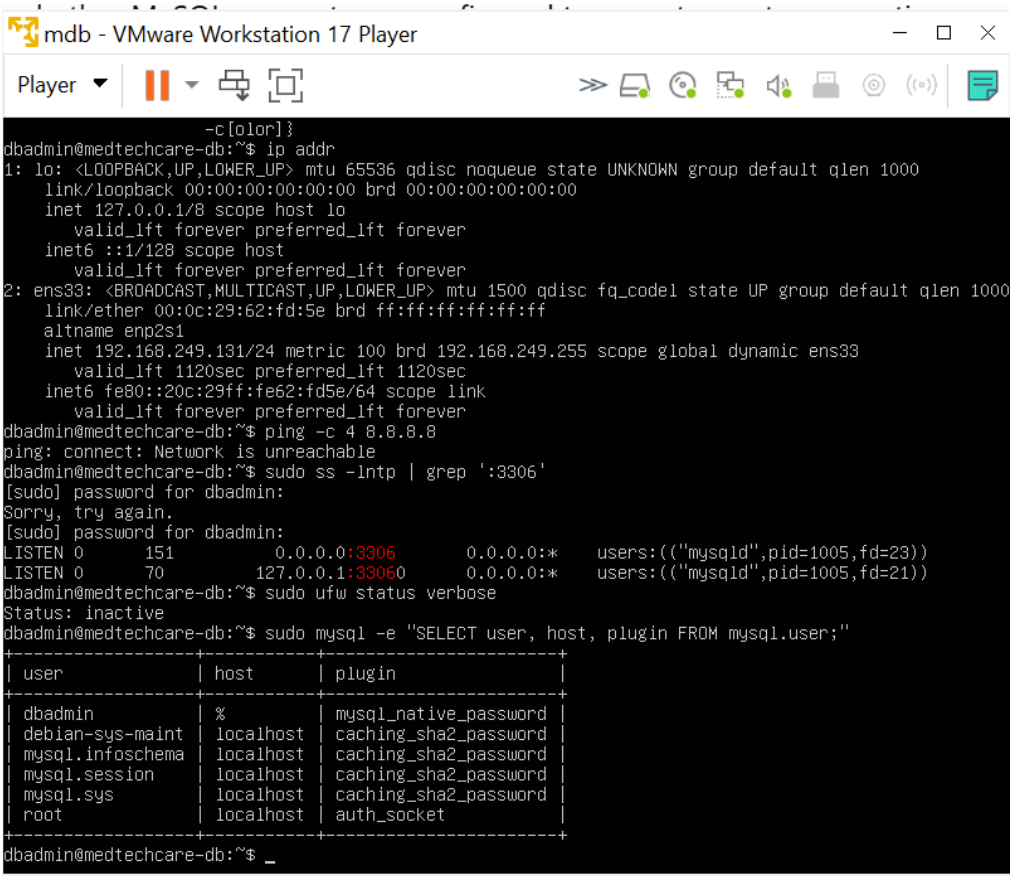


Figure 13. MySQL account host restrictions and authentication plugins.

Interpretation. The wildcard pattern means the account can attempt authentication from reachable hosts (subject to credentials and other controls). This account was created for the MSc PoC, not discovered unexpectedly in a production database.

Why next? Inspect the privileges that a successfully authenticated dbadmin session would possess.

Step 15 — Verify privileges assigned to the lab account

Purpose. Determine the impact that access via the broad-host account could have on the named lab databases.

```
sudo mysql -e "SHOW GRANTS FOR 'dbadmin'@'%';"
```

Observed result. The account had USAGE at the global scope and ALL PRIVILEGES on two named MedTechCare laboratory databases; the grants did not show global ALL PRIVILEGES.

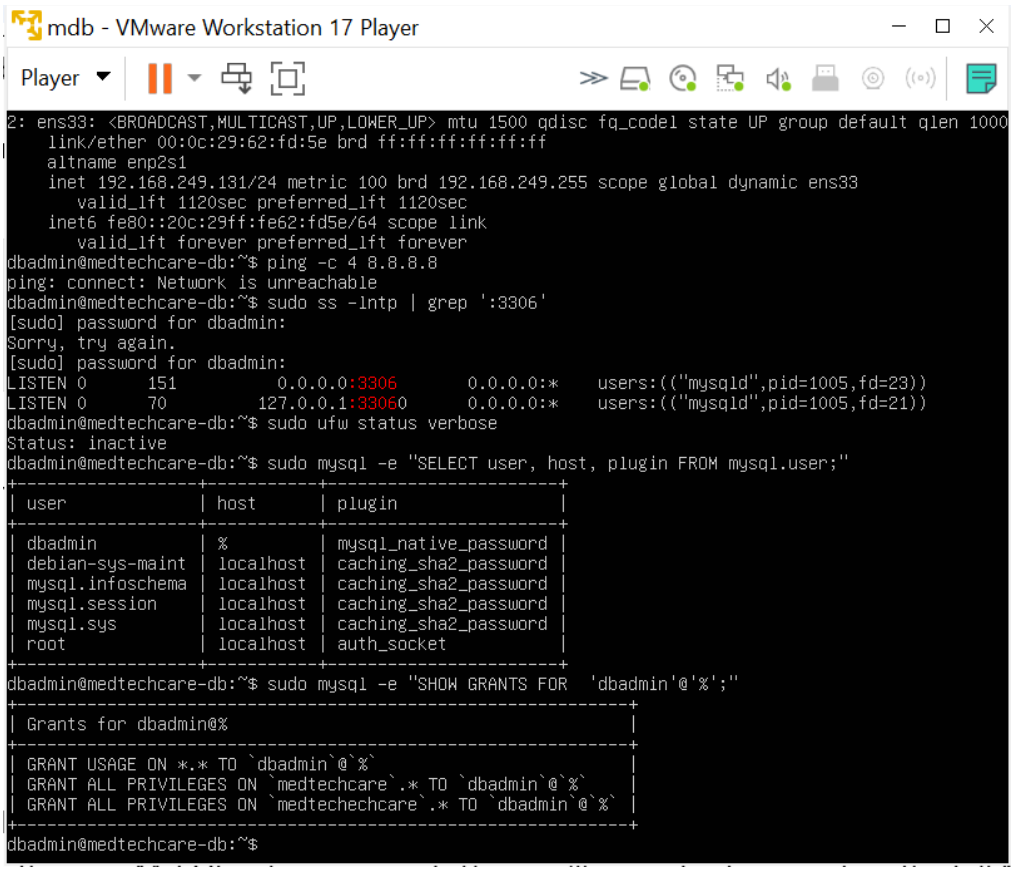


Figure 14. Read-only MySQL privilege verification for the intentionally configured PoC account.

Interpretation. The account has extensive power within the named databases, but this alone does not establish server-wide administrator rights or unauthorised access. Risk would depend on network reachability, credential compromise, privileges and data sensitivity.

Why next? Consolidate confirmed observations into risk findings rather than claiming vulnerabilities not demonstrated.

Step 16 — Record the intentional network-isolation limitation

Purpose. Explain why attempts to refresh the Ubuntu package index could not verify whether later security updates were pending.

```
sudo apt update
```

Observed result. The Ubuntu VM could not resolve Ubuntu archive domains; a direct ping to 8.8.8.8 returned “Network is unreachable”. This was consistent with the deliberately disconnected lab network, not an unexpected loss of production connectivity.

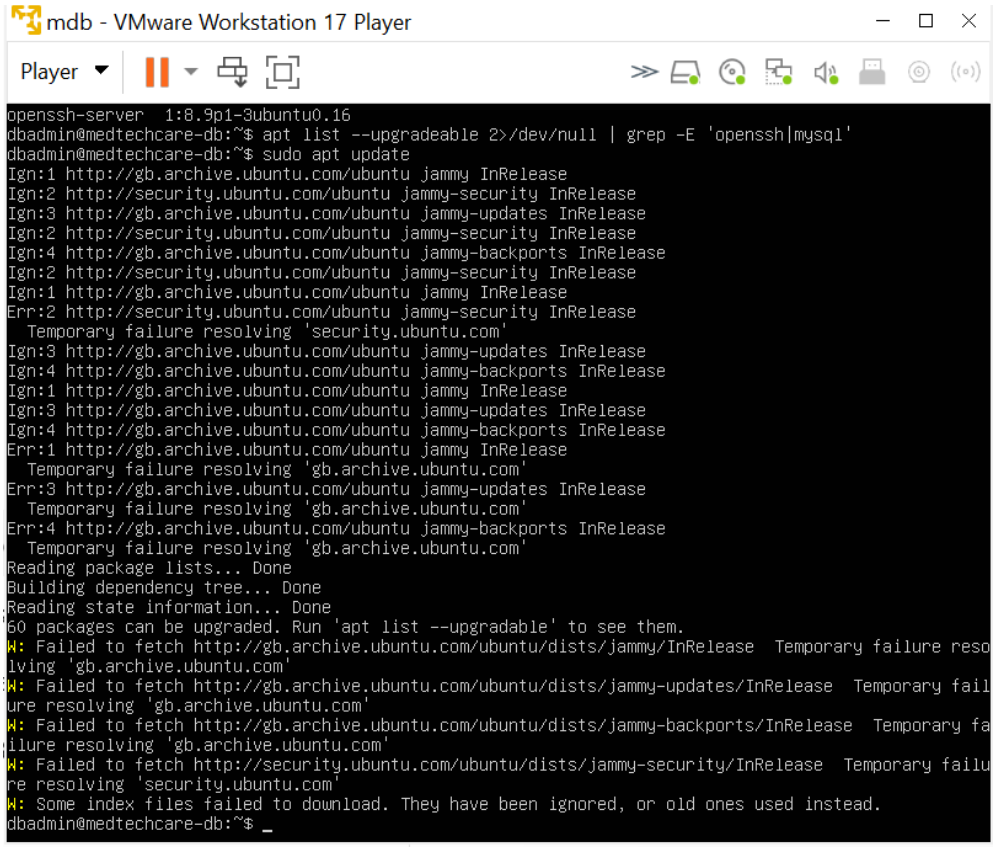


Figure 15. APT could not refresh repository information because the lab was isolated.

Interpretation. Local APT lists were stale, so the audit cannot claim that the machine has the latest SSH or MySQL updates. The CVE-2024-6387-specific conclusion remains grounded in the installed package and Ubuntu's documented fixed version.

3. Findings and risk prioritisation

Risk ratings are contextual judgements for a small isolated training network. They should not be interpreted as validated production-impact measurements. No unauthorised database access, password bypass or remote code execution was demonstrated.

ID	Observation and evidence	Assessment
MY-01	MySQL 3306 reachable; 0.0.0.0 listener; UFW inactive; dbadmin@% has ALL PRIVILEGES on two named lab databases.	Confirmed lab configuration; conditional Medium exposure in a general deployment, potentially higher impact if sensitive data/weak credentials exist.
SSH-01	OpenSSH 1:8.9p1-3ubuntu0.16 versus Ubuntu CVE-2024-6387 fixed revision 1:8.9p1-3ubuntu0.10.	CVE-2024-6387 addressed in installed Ubuntu package; not a reportable unpatched finding for this CVE.
SSH-02	Advertised SHA-1-based SSH MAC options alongside stronger alternatives.	Policy-dependent hardening observation; no exploit demonstrated.
OBS-01	MySQL empty-password script returned no positive match.	Negative/inconclusive check, not proof of strong authentication.
LIMIT-01	VM isolated from internet; apt update unsuccessful.	Latest security update status not determined.

4. Recommendations and verification steps

- **Restrict database reachability.** If remote MySQL connections are not needed, bind only to loopback; otherwise apply network filtering so only approved client addresses can reach TCP/3306.
- **Scope MySQL account hosts.** Replace wildcard remote access patterns with specific approved hosts or ranges where practical. Keep strong, unique credentials and review authentication configuration.
- **Apply least privilege.** Review whether the lab application needs ALL PRIVILEGES on the two databases; grant only required permissions in real deployments.
- **Maintain security update hygiene.** For non-isolated deployments, regularly refresh signed package metadata and install vendor security updates. Use Ubuntu advisories to assess package revisions rather than upstream strings alone.
- **Review SSH cryptography policy.** Assess whether SHA-1 MAC compatibility is actually required; prefer vetted modern algorithms when clients support them.
- **Reassess after changing controls.** Repeat targeted Nmap checks and host-side review to confirm that disallowed clients cannot reach or authenticate to MySQL.

5. Ethical and technical limitations

This assessment is limited to the two controlled VMs and a subset of protocols and controls. The 65,535-port result is TCP-only; UDP and full application-level penetration testing were out of scope. Nmap fingerprints and NSE results cannot independently demonstrate complete vulnerability or absence of vulnerability. Only synthetic MedTechCare training data was used; no actual patient records or third-party systems were assessed. The MySQL broad-access configuration was intentionally set up for an earlier educational PoC. No claim of exploitation, database breach, privilege escalation or leaked credentials is made.

Figures are cropped evidence from the genuine VMware/terminal sessions. Public-release copies omit the transient MySQL handshake salt and exclude surrounding browser or unrelated desktop content. The original unmodified scan outputs and screenshots are retained privately for evidence integrity.

6. Conclusion

The investigation followed a defensible chain of evidence: identify the lab address, discover hosts, enumerate the target's TCP ports, fingerprint exposed services, test narrow service properties, validate the findings locally and assess their practical significance. Vendor correlation prevented a false positive for CVE-2024-6387, while host-side checks established how MySQL was deliberately exposed in the MedTechCare proof-of-concept environment. The exercise demonstrates how penetration-testing output becomes more useful when interpreted through deployment context, likelihood and business impact.

Appendix A — Original scan files and provenance

Primary Nmap evidence was supplied from ~/nmap-network-scanning-lab/scans/, collected 7–8 October 2026 using Nmap 7.95. Seven public normal-output files accompany this report; the transient MySQL handshake salt was redacted in the public copy. The original seven outputs are preserved privately and were not altered.

Original filename	Assessment phase
host-discovery.txt	Network host discovery
full-tcp-scan.txt	All TCP port scan
service-version-detection.txt	SSH and MySQL version detection
os-detection.txt	OS fingerprinting

ssh-enumeration.txt	SSH supported algorithms
mysql-info.txt	MySQL handshake metadata
mysql-empty-password.txt	Limited empty-password script result

Note: the initial default TCP scan and some local command checks are documented by screenshots rather than separate -oN .txt files. Absence of a separate scan text file is not filled with reconstructed or fabricated output.

Appendix B — External references

Ubuntu Security Team (2024), CVE-2024-6387. https://ubuntu.com/security/CVE-2024-6387 (accessed 17 October 2026).

Ubuntu Security Team (2024), USN-6859-1: OpenSSH vulnerability. https://ubuntu.com/security/notices/USN-6859-1 (accessed 17 October 2026).

Nmap Project, Nmap Reference Guide and NSE documentation. https://nmap.org/book/man.html and https://nmap.org/nsedoc/ (accessed 17 October 2026).